



Algorithm & C-Programming

Project work 2025:
Chess in C

For the engineering school: ECE-Paris



Info

Edition:

1.4

 (2025.11.25)

Written by: Simon Céleste VANDAMME

svandamme@omnesintervenant.com



01. FOREWORD

A. General information

This project can be done by yourself, or in a group up to 4 people .
It should take about 3-7 days of work to get completed.

C. AI: RECOMMENDATIONS and DOWNSIDES

Artificial Intelligence (AI) tools show remarkable results, especially in the field of programming. Their integration into most Integrated Development Environments (IDEs) nowadays makes them more accessible than ever.

Used wisely, AI can be a valuable support in your journey to learn C programming. However, it can also be tempting to rely on it to complete entire projects or exercises in seconds.

When learning a new skill in any subject, remember this golden rule:

UNDERSTANDING > REWRITING PRE-DONE CONTENT

Note that I am NOT AGAINST the use of AI in your works.

However, I would highly appreciate it if you were transparent upon using it in your code and specify EVERY TIME why you are relying on it and how the snippet code works.

RECOMMENDATIONS

01. First think about the subject YOURSELF, write some elements down.
Then, use AI to continue your work if needed.

Your learning suffers when you let AI solve problems before trying yourself.
Skip the thinking, and you only train dependency... not skill.

As future engineers, you are expected to understand problems deeply and think creatively.
Keep training those skills: they are the foundation of your value as an engineer.

02. Use AI tools as personal teachers.
They are great to highlight your mistakes or improve your code quality.

Once done with a working program, you can prompt the AI on what is to be improved, for example:

- How to secure your code against errors?
- Ask for alternative ways to solve the problem. Did you use the most interesting way?
- How to improve your memory management or time execution?

03. YOU are responsible for the code you submit... not the AI.

Whether at school or in your future job, your program is your creation. You must fully understand it and be able to explain every part of it, even if you received significant help from AI.

AI has limits: it struggles with complex projects, can produce insecure code, and lacks real creativity.
As an engineer, it is your job to go beyond these limits.

02. Introduction

Chess is a classical strategy game and an excellent programming challenge.

Over four weeks, you will create a progressively more complete chess program using only the C tools you currently know: variables, arrays, loops, functions, and simple text I/O.



You will begin with a simple text-based version and progressively add more advanced features, such as move validation, game rules, save system, and **optionally** a graphical interface.

This project must be developed **in C**, using **standard libraries only** until the optional graphical version.

Project Objectives

By the end of the project, you should be able to:

- Design and create a system for **user input** with conditions.
- Experiment with **modular** programming (header **.h**, code **.c**, saving file...).
- Model the game board, pieces, and rules.
- Have an **AGILE approach** (incremental development), with recommended “sprint” (program version) and set features.
- OPTIONALLY: use of a Git solution, such as GitHub (*even if done without a team*)
- OPTIONALLY: interface C-code with a **graphical library** of your choice.

03. How Chess Works

Chess is a strategic board game played by **two players**.

Each player controls a set of pieces placed on an **8×8 board**, and players take turns performing exactly **one action** per turn. An action usually means **moving one piece** according to its movement rules.

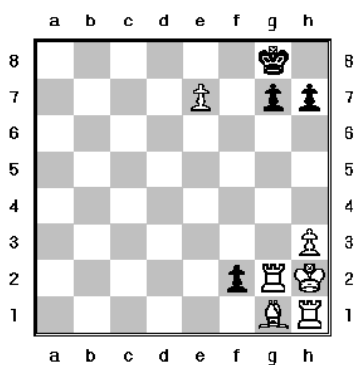
The game alternates strictly:

- **White plays first**
- **Black plays next**
- ... And so on, until the game ends

If you don't know what the rules of chess are, here are two links I recommend from major website:

- **“Rules of Chess”**, Wikipedia
https://en.wikipedia.org/wiki/Rules_of_chess
- **“Chess Pieces”**, Chess.com
<https://www.chess.com/terms/chess-pieces>

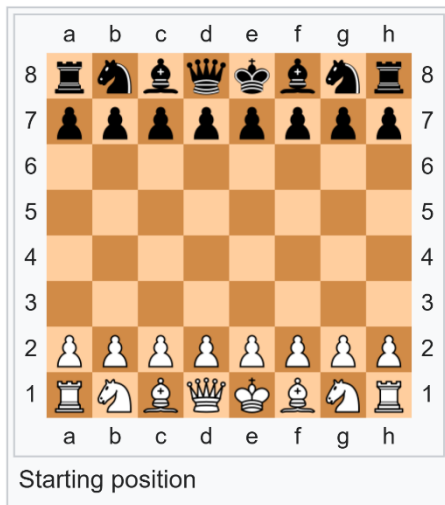
Board Layout



- Chess is played on an **8×8 grid**.
- Columns: **a → h**.
- Rows: **1 → 8**.
- Each cell is written like: e2, a7, h1, etc.



Pieces and Their Movement



Each player has:

- **1 King:** moves 1 square in any direction.
- **1 Queen:** moves any number of squares in any direction.
- **2 Rooks:** move horizontally or vertically.
- **2 Bishops:** move diagonally.
- **2 Knights:** move in “L”-shape (2+1).
- **8 Pawns:** advance forward, capture diagonally.

Special rules (for the [V3 chess code](#)):

- Pawn promotion
- Castling
- “En passant”, with the pawns.

Victory conditions:

1. Checkmate ends the game: the king is threatened by a piece and cannot be saved anymore by any possible move (in 1 turn).

2. Stalemate results in a draw.

04. Global Requirements

This first version focuses entirely on a **text-based implementation**, displayed in the **terminal**.

From **Version 1 up to Version 3**, your chess program will not use any graphical library. Everything must be handled through console output and user input.

The chessboard must be displayed with clean and readable coordinates on both axes:

- Columns labeled from **a** → **h**
- Rows labeled from **1** → **8**

Example of a rendering of your board (done with `printf()...`):

	a	b	c	d	e	f	g	h	
8	r	n	b	q	k	b	n	r	8
7	p	p	p	p	p	p	p	p	7
6									6
5									5

Pieces must be represented using **single characters**:

- **White pieces:** uppercase (K, Q, R, B, N, P)
- **Black pieces:** lowercase (k, q, r, b, n, p)

Empty squares can be displayed as space or a dot.



Regarding the user inputs, for all text-based versions (V1 to V3), the interaction model must follow a simple input procedure:

1. **Ask the user to select a piece to move**

2. Select piece to move?

3. **Ask the user for the destination square**

Input format:

4. Select destination: e4

5. >> The program processes the move:

- **Version 1:** Move is executed even if illegal (no rules enforced)
- **Version 2:** Program must verify the move is legal for that piece
- **Version 3:** Moves should respect rules *and* detect special game states (like capturing a king)

Remember: it's all made with SCANF()... no other input forms (arrow keys, menus...) are expected at this stage.

Here is an **example** of how the user input could be handled:

```
Select a piece on the board?  
Piece: e3  
  
You've selected the 'KNIGHT' (white).  
Where do you want to put it now? ('undo' or 'u' to get back)  
Destination = f4
```

05. Grading & Roadmap

This is the core of the project.

Each version is a stable milestone with increasing complexity.

V1: Minimal Chess Engine (8 points)

Goal of this version: “Basic playable board WITHOUT move validation.”

Overall features:

- Board display
- Basic input/output
- Piece movement (unrestricted)
- Turn alternation

Details:

- Display the board.
- Initialize all pieces in correct positions.
- Ask the user for:
 - Origin coordinate
 - Destination coordinate
- Move the piece **without checking legality**.
- Switch turns between White and Black.
- Basic error checking:
 - Coordinates exist (a–h, 1–8)
 - A piece exists on the selected origin



V2: Legal Move Validation (7 points)

Goal: “Moves must follow real chess rules.”

Overall features:

- Fully functional rules.
- Illegal moves prevented with error messages.
- Clear separation of rule-checking functions.

Details:

- Implement movement rules for each piece, such as:
 - Straight/diagonal line movement
 - Knight jump
 - Pawn forward movement and diagonal capture
 - ... (everything that follows the chess rules!)
- Collision detection:
 - Cannot jump over pieces (except Knight)
- Turn-based color restriction:
 - White moves white pieces, etc.
- Detect **check** and prevent illegal king exposure (optional for V2 / mandatory for V3).
- Winning conditions, your game detects:
 - Check
 - Captured king
 - Checkmate (optional for V2 / mandatory for V3)



V3: Game State Management (5 points)

Goal: “Add persistence and quality-of-life features.”

Overall features:

- A main menu, that greets the user
- .txt or .bin save file.
- Functions for serializing/deserializing the board.
- Documentation explaining save format.

Details:

- Main menu
 - With choices like: start a game, load a game, (eventual) settings, quit the game...
 - Input taken with scanf: “1” = start, “2” = load, (...).
- Save current board to file.
- Load a saved game.
- Store a list of moves (log file).
- Track:
 - Number of moves
 - Captured pieces
- Optional:
 - Leaderboard (games won / lost)
 - Player names



V4: Optional GUI Extension (++ extra points)

Goal: “Display the game with a graphical library.”

Possible Libraries

- SDL3
- Raylib
- SFML (C-compatible)

Some features we could expect:

- Render board graphically.
- Mouse-selected pieces.
- Having a starting screen, with clickable options.
- Visual feedback on valid moves (hovering piece?)
- ...

You could also add new features or game modes, such as:

- Randomized chess.
- Chess with fog of war.
- basic AI.
- ...

Essentially, everything that feels “extra” to the project.

Just as a REMINDER: this version is NOT mandatory.

Do it if you want to get your hands on graphical interfaces, with popular libraries.

But everything in this version only earns you extra points, and the first versions (v1-v3) are a priority before everything else. :)

06. Evaluation Criteria

Your final grade will be based not only on the technical quality of your code, but also on your ability to present, explain, and deliver the project as a complete, organized work.

At the end of the assignment, you must provide **2 elements**:

Deliverable	Description
Full Project (.c, .h, .cpb ...)	Complete, runnable Code::Blocks project with all source code
15-minute Oral Presentation	Technical and team-focused presentation with demo
2-minute Video (optionnal)	Short demonstration for bonus points

1. Full Project Delivery (mandatory)

You must submit the entire working project, in a form that can be opened and run directly in **Code::Blocks**.

Your delivery must include:

- **Complete Source Code**
 - All .c and .h files.
 - A clean folder structure.
 - A main.c that compiles and runs without errors or warnings.
 - Clear comments and fixed-point indentation.
 - No unused files, no temporary code.
- **Build Instructions**
 - A short README.txt or README.md containing:
 - How to compile the program
 - How to launch it
 - If needed, extra explanations (files created, save format, etc.)
- **Project Integrity**
 - Your program must compile without manual intervention.
 - Your program must run in a standard environment (Windows or Linux).
 - If your project requires additional files (assets, save files, configs), include them.



2. Oral Presentation – 15 minutes (mandatory)

You will present your project as a **team**, for **15 minutes**, followed by a short question and answer. I'll be asking you to give me the presentation (PowerPoint / PDF support).

Regarding what your presentation must cover:

- **The Team Workflow**
 - How you organized the work
 - How responsibilities were split
 - Tools used (Git, Trello, Discord, etc.)
- **Technical Overview**
 - High-level structure of your program
(e.g., board management, movement functions, rule validation...)
 - How you implemented key features
 - What was challenging and how you solved it, what you learned...
- **Demonstration**
 - Showing the program running
 - Demonstrating:
 - Board display
 - Typical moves
 - Error handling (for V2)
 - Saving/loading (for V3)...
- **Retrospective**
 - What went well
 - What you would improve
 - What features you would add with more time

This presentation is graded on clarity, structure, technical understanding, and teamwork.

3. Optional Video Demonstration – 2 minutes (bonus)

This is not mandatory, but can grant **extra points**.

You may record a **short 2-minute demonstration video** showing your program in action.

Minimum expectations:

- Show the board in the terminal
- Perform a few moves
- Demonstrate a feature specific to your version (e.g., illegal move detection, saving a game)

Recommended tools:

- **OBS Studio** (if you know how to use it)
- Windows Game Bar (Win+G)
- macOS screen recording
- ... Otherwise, if nothing works in your favor: a recording with your phone can be accepted.

Quality Notes:

- Audio commentary is optional
- No need for editing or fancy production—clarity matters more than style



07. TECHNICAL SPECIFICATION SHEET

A. DOCUMENT SPECIFICATIONS

TECHNICAL INFORMATION	
Document Version	1.4
Release Date (YYYY.MM.DD)	2025.11.25
Course Title	C-LANGUAGE & ALGORITHM
Chapter Name	C-CHESS – Project
Document ID	PROG-C-PROJECT-CHESS
Keywords	C, IDE, program, worksheet, course materials, digital
File Name	C-TD-ChapterXX-v1.0.pdf
Page Count	16 pages
Contributors	▪ Nathalie AMSALLEM: Proofreader ▪ Léo BELUITE: Proofreader ▪ Drazic MARTIN: Proofreader and program tester ▪ Simon Céleste VANDAMME: Author
Maintainer Contact	celeste.vandamme@outlook.com
Target Audience	Designed for engineering students starting to learn the C-programming language.
Languages Available	▪ English: available ▪ French: not yet available
Answer Sheet	Not yet available
Confidentiality Level	Public
License	Creative Commons Attribution 4.0 International (CC BY 4.0) Link: https://creativecommons.org/licenses/by/4.0/



VERSION HISTORY			
Version	Release Date (YYYY.MM.DD)	Authors	Description / What is new in the version?
1.0	2025.09.27	– VANDAMME S.	Official release of the document.